

数据库

1, 深度分页慢因及优化

深度分页慢，主要是因为 `LIMIT offset` 会先扫描前面的大量数据，即使最终只取几行，数据库也必须先跳过 `offset` 行，导致扫描行数过多，性能下降。

优化方式主要有两种：

1. **游标分页**：用上一页的最后一条 ID 往后查，避免 `offset`，性能最好。`select * from table where id > 100000 limit 20`
2. **索引覆盖**：先通过索引查 ID，再回表拿数据，减少扫描量。

2, 分页为何配ORDERBY

分页必须配合 `ORDER BY`，是为了保证数据顺序稳定，否则分页结果可能重复或丢失；同时建议 `ORDER BY` 字段使用索引，避免深度分页性能问题。

3, 索引覆盖优化分页原理

先用子查询在索引中完成排序和分页，避免大量回表；再用主键一次性回表取数据，从而减少 IO 和扫描行数，提高性能。

4, 游标与LIMIT分页对比

`LIMIT` 分页通过 `offset` 跳过数据，`offset` 越大扫描行数越多，深度分页性能差；`LIMIT offset, size`。

游标分页通过上一页最后一条 ID 向后查询，不跳过数据，只扫描需要的行，性能稳定，更适合大数据量场景。`WHERE id > last_id`

5, 索引查询更快的原因

索引基于 B+Tree，查找时只需少量磁盘 IO，避免全表扫描；叶子节点有序存储，支持高效点查和范围查询，配合覆盖索引还能避免回表，因此查询更快。

6, 生产少用RIGHT JOIN原因

RIGHT JOIN 可读性差、优化弱、可替代，LEFT JOIN 更直观、更稳定。

7，三种JOIN区别

INNER JOIN 只返回两表都满足条件的数据；

LEFT JOIN 返回左表全部数据，右表无匹配则为 NULL；

RIGHT JOIN 返回右表全部数据，左表无匹配则为 NULL；

实际生产中多用 INNER JOIN 和 LEFT JOIN，RIGHT JOIN 一般被 LEFT JOIN 替代。

8，慢查询排查优化

慢查询排查先看慢日志，再用 EXPLAIN 分析执行计划，重点检查是否走索引、扫描行数和排序方式，通过建索引、改写 SQL 和业务优化解决

9. 常见索引类型有哪些

- **主键索引**：主键自带，唯一且非空，最快
- **唯一索引**：列值唯一，允许一个 NULL
- **普通索引**：最基础，仅加速查询
- **联合索引**：多列组合，遵循最左匹配
- **覆盖索引**：查询字段全在索引里，无需回表

Linux

1，删除文件空间未释放原因

文件被删除，但仍被某个进程占用（打开），只有当进程关闭文件或退出，才会真正释放磁盘空间。

2，清空被占用日志

清空被占用日志应用 `> app.log` 而不是 `rm`，可立即释放磁盘空间且不影响持锁进程继续写入

3，find 命令其他用法

find 可按文件名、类型、大小、修改时间查找；常用在日志清理，如查找 N 天前大文件并删除或清空。find /data -name "*.log"

find . -type f

4，Linux 文件权限及修改

改权限：chmod，改归属：chown

网络

1，被动方不能同时发FIN+ACK原因

TCP 是双工通信，收到 FIN 只表示对方不再发送数据，并不代表自己也不再发送；所以被动方要先 ACK（确认关闭读端），等本端数据发完再发 FIN，不能合并。

2，四次挥手状态变化

主动关闭方发 FIN 进入 FIN_WAIT_1，收到 ACK 进入 FIN_WAIT_2，收到对端 FIN 进入 TIME_WAIT 并等 2MSL 后关闭；

被动关闭方收到 FIN 进入 CLOSE_WAIT，发 FIN 后进入 LAST_ACK，收到 ACK 才 CLOSED。

3，等待2MSL原因

2MSL 是主动断开方（TIME_WAIT）等待的时长，核心两个原因：

1. **确保被动方收到最后 ACK**：防止 ACK 丢包，被动方重传 FIN，2MSL 内可重发 ACK，避免连接残留。
2. **防止旧连接报文干扰新连接**：让网络中旧连接的滞留报文彻底消失，避免新连接收到旧数据，保证可靠性。

4，SYN/ACK/FIN报文作用

- SYN：发起连接请求，**同步序号**，三次握手第一步用。
- ACK：**确认收到数据 / 请求**，带确认号，握手、挥手、传数据都要用。
- FIN：**请求关闭连接**，表示本方数据发完，四次挥手用。

5, TIME_WAIT含义及作用

含义：主动断开连接方，发完最后 ACK 后进入的**等待状态**，持续 2MSL。

作用两点：

1. **可靠关闭：**确保被动方收到 ACK；ACK 丢了，2MSL 内可重发，避免被动方一直重传 FIN。
2. **防旧报文干扰：**等网络里旧连接残留包消失，防止新连接收到旧数据，保证安全。

脚本

1, 断言优缺点及生产环境使用

优点：开发调试快速，自动检查关键条件，快速暴露 bug，代码轻量。

缺点：默认关闭不生效，失败直接崩溃进程，不适合处理可预期异常。

生产环境：绝对不建议用断言处理业务逻辑。

生产要用if **判断** + **异常捕获** + **日志记录**，保证服务不崩、可监控、可容错。

2, os.path.join 跨平台原理

os.path.join 会根据当前操作系统自动选择路径分隔符，并按规则拼接路径，避免硬编码分隔符导致的跨平台问题。

3, Shell 异常处理

Shell 默认不中断执行，需要显式设置错误检测与退出机制。

4, Python检查文件存在/判空

```
import os
# 1. 检查文件是否存在
os.path.exists("test.txt")    # 存在返回True
os.path.isfile("test.txt")    # 确认是文件（不是目录）
# 2. 判断文件是否为空
os.path.getsize("test.txt") == 0 # 大小=0 就是空文件
```

5，相对/绝对路径优缺点

相对路径可移植但依赖运行环境，绝对路径稳定但不够灵活；生产环境推荐用绝对路径，或基于脚本所在目录动态构造绝对路径。

6，Python异常处理机制

```
try:
    # 可能出错的代码
    do_something()
except ValueError as e:
    # 处理指定异常
    print(e)
else:
    # 没有异常时执行
    print("success")
finally:
    # 无论是否异常都会执行
    print("cleanup")
```

Python 通过 try/except 捕获异常，else 处理无异常分支，finally 保证资源释放；生产环境应捕获具体异常并记录日志，避免程序无声失败。

快速排序

```
def quick_sort(arr):
    # 递归终止条件
    if len(arr) <= 1:
        return arr
    # 选择基准
    pivot = arr[0]
    # 分区：小的放左，大的放右
    left = [x for x in arr[1:] if x <= pivot]
    right = [x for x in arr[1:] if x > pivot]
    # 递归 + 合并
    return quick_sort(left) + [pivot] + quick_sort(right)
```

```
def quick_sort(arr, left=None, right=None):
    if left is None:
        left = 0
    if right is None:
        right = len(arr) - 1

    def partition(l, r):
        pivot = arr[r] # 选最右为基准
        i = l - 1
        for j in range(l, r):
            if arr[j] <= pivot:
                i += 1
                arr[i], arr[j] = arr[j], arr[i]
        arr[i+1], arr[r] = arr[r], arr[i+1]
        return i + 1

    if left < right:
        p = partition(left, right)
        quick_sort(arr, left, p - 1)
        quick_sort(arr, p + 1, right)
```